

Contents

Part I — Foundations	5
1 Introduction	5
1.1 Purpose of UF	5
1.2 Conceptual Boundary of UF	6
1.3 Structure of UF–Spec v1.4.0	6
1.4 Relationship to Domain Adapters	7
1.5 Compliance Principle	7
2 Mathematical Overview	7
2.1 Sequential Input Domain	8
2.2 Layered Operator Model	8
2.3 Structural Evaluation Vectors (SEV)	8
2.4 Gate Segmentation (L1) Overview	9
2.5 Interpretive Layer (L2) Overview	9
2.6 Resonance Engine (L3) Overview	9
2.7 Decision State Fields (L4) Overview	9
2.8 Summary of Structural Flow	10
3 Structural Evaluation Vectors (SEV)	10
4 Gate Segmentation and Mosaic Architecture	12
5 Interpretive Structural Metrics	13
6 Resonance Engine	14
7 Decision Dynamics	15
Part II — Global Properties	17
8 Invariants	17
8.1 Deterministic Operator Invariance	17
8.2 Invariance to Null Structural Transforms	17
8.3 Gate Boundary Invariance	18
8.4 Interpretive Layer Invariance	18
8.5 Resonance Invariance	18
8.6 DSF Invariance	19
8.7 Version Invariance	19
8.8 Summary of Invariants	20
9 Stability Metrics	20
9.1 Perturbation Model	20
9.2 Gate Stability Score (GSS)	21
9.3 Directional Stability Score (DSS)	21

9.4 DSF Stability Score	22
9.5 Composite Structural Stability Score $S(UF)$	22
9.6 Robustness Score $R(UF)$	22
9.7 Relationship Between Stability Metrics and Invariants	23
9.8 Summary of Stability Metrics	23
10 Hardening Model (Kernel Safety)	23
10.1 Hardening Overview	24
10.2 Hardening Trigger Conditions (HTCs)	24
10.3 Hardening Actions (HAs)	25
10.4 SafeMode	26
10.5 Hardening Recovery Rules (HRRs)	26
10.6 Traceability	27
10.7 Summary	27
11 Formal Mathematics Framework	28
Part III — Validation	29
12 Academic Validation Layer	29
12.1 Purpose of Validation	29
12.2 Validation and the UF Kernel Boundary	30
12.3 Perturbation as a Structural Probe	30
12.4 Collapse Modes	30
12.5 Reproducibility Requirements	31
12.6 Validation as the Academic Foundation of Adapters	31
12.7 Summary of Academic Validation	31
13 Validation & Benchmarking Suite	32
13.0 Purpose	32
13.1 Validation Concepts	32
13.1.1 Global Stability Functional	32
13.1.2 Robustness Index	32
13.1.3 Structural Validation Domains	32
13.2 Core Validation Requirements	32
13.3 Noise & Perturbation Robustness	33
13.4 Adversarial Structured Input Tests	33
13.5 Parameter Perturbation Tests	33
13.6 Memory Load & Schema Stress Tests	33
13.7 Multi-Lattice Conflict Tests	33
13.8 Breathing-Self-Organization Synchronization	33
13.9 SES Evolution Tests	33
13.10 Cross-Domain Transfer Tests	33
13.11 Global Stability Shock Tests	34
13.12 Validation Summary	34
13.0.0 Global Protocol Standards	34

13.3.1 Noise and Perturbation Robustness	35
13.4.1 Adversarial Structured Input Tests	38
13.5.1 Parameter Perturbation Tests	40
13.6.1 Memory Load and Schema Stress Tests	43
13.7.1 Multi-Lattice Conflict Tests	46
13.8.1 Breathing-Self-Organization Synchronization	48
13.9.1 SES Evolution Tests	51
13.10.1 Cross-Domain Transfer Tests	53
13.11.1 Global Stability Shock Tests	56
Validation Matrix	59
Part IV — Adapters	59
14 Adapter Architecture Overview	59
15 SES-Core (UF Encryption Platform)	60
C.1 Purpose and Scope	60
C.2 Terminology and Definitions	60
C.3 Threat Model	61
C.4 UF Structural Payload	61
C.5 Redaction Policy	61
C.6 SES Envelope Format	62
C.7 Two-Layer Key Architecture	62
C.8 Structural DRBG	62
C.9 AEAD-SIV Encryption	62
C.10 Algorithms	62
C.11 Domain Binding Rules	63
C.12 Security Properties	63
C.13 Implementation Requirements	63
16 Financial Domain Adapter	64
D.0 Purpose and Scope	64
D.1 Financial Input Domain Specification	64
D.2 UF Invocation Rules in the Financial Domain	65
D.3 TFE-Side Summary Requirements	65
D.4 Multi-Asset and Multi-Class Support	66
D.5 Domain Binding Requirements in the Financial Adapter	67
D.6 SES-Core Usage Requirements in the Financial Domain	67
D.7 Governance Layer (TFE) Interpretation Logic	67
D.8 Performance and Deployment in Market Systems	68
D.9 Validation Requirements for the Financial Adapter	68
D.10 Appendix D Completeness and Specification Closure	69
17 Future Adapters	69
Part V — Governance	69

18 Versioning Rules	69
18.1 Purpose of Versioning	70
18.2 Semantic Versioning Model	70
18.3 Kernel Version Isolation	70
18.4 SES–Core Version Compatibility	70
18.5 Adapter Version Compatibility	71
18.6 Required Version Increments	71
18.7 Summary	72
19 Compliance & Reproducibility	72
19.1 Deterministic Behavior Requirement	72
19.2 Version Isolation Requirement	72
19.3 Validation Conformance Requirement	73
19.4 SES–Core Compliance Requirement	73
19.5 Domain Adapter Compliance Requirement	74
19.6 Reproducibility Requirements	74
19.7 Auditability Requirements	74
19.8 Compliance Summary	75
20 Glossary	75
21 References	76
21.1 Normative References	76
21.2 Informative References	77
21.3 UF Internal Cross-References	77
21.4 Reference Format	77
21.5 Completeness of References	78
22 Governance and Versioning Rules	78
22.1 Purpose of Governance	78
22.2 Versioning Policy	78
22.3 Canonical Implementation Requirements	79
22.4 Change Control	79
22.5 Forbidden Behaviors	80
22.6 Governance of Adapters	80
22.7 Reproducibility Obligations	80
22.8 Governance Summary	81
Figures	81
Changelog	81

Part I — Foundations

1 Introduction

The Unified Framework (UF) is a deterministic, multi-layer structural analysis system designed to evaluate sequential data by transforming an input series through a tightly defined chain of operators. UF differs fundamentally from predictive or probabilistic models: it does not generate forecasts, does not infer latent variables, and does not produce or rely on stochastic estimates. Instead, UF measures and characterizes the internal structure of an input series through mathematically constrained operators (Layers 0–4), each of which produces a well-defined set of structural quantities.

The UF Kernel is composed of five layers:

- **L0** — Structural Evaluation Vector (SEV) construction,
- **L1** — Gate Segmentation,
- **L2** — Interpretive Layer,
- **L3** — Resonance Engine,
- **L4** — Decision State Fields (DSF).

Each layer is fully specified, deterministic, and reproducible. No layer modifies or reinterprets its inputs beyond the rules defined in UF-Spec v1.4.0. As a consequence, two independent implementations of the UF Kernel MUST produce bit-identical structural outputs whenever their inputs and configuration parameters match.

1.1 Purpose of UF

The purpose of UF is to measure structural behavior in sequential systems where qualitative or emergent phenomena cannot be captured directly through statistical or predictive modeling. UF does not attempt to answer “What will happen next?” Instead, it answers:

- How stable is the structure contained in the input?
- How robust is this structure to perturbation?
- Are there signs of structural collapse or phase transition?
- Does the system exhibit coherent or incoherent dynamics?

UF provides a unified grammar for describing all such behavior. This makes the system applicable not only to financial time series (via Part IV adapters), but also to scientific, engineering, and cognitive-domain analysis (via future adapters defined in Section 17).

1.2 Conceptual Boundary of UF

UF is explicitly *not* a learning system, inference system, probabilistic model, or optimization engine. It possesses:

- no training process,
- no fitted parameters,
- no internal state carryover between evaluations,
- no adaptive feedback or update mechanism.

Each evaluation of UF is a closed operation: inputs produce structural outputs, and the system resets to a blank state after completion. This property ensures:

- reproducibility,
- auditability,
- version isolation,
- deterministic clarifiability of all UF behavior.

1.3 Structure of UF–Spec v1.4.0

UF–Spec v1.4.0 is organized into five parts:

- **Part I** — Kernel Foundations,
- **Part II** — Global Properties,
- **Part III** — Validation,
- **Part IV** — Domain Adapters,
- **Part V** — Governance and Compliance.

Part I defines the core Kernel operators and mathematical framework. Part II describes invariants, stability metrics, kernel safety, and formal mathematics. Part III defines the validation suite and benchmarking protocols. Part IV defines domain-specific adapters, including the SES–Core encryption platform and the Financial Adapter. Part V defines versioning rules, compliance requirements, and reproducibility obligations.

1.4 Relationship to Domain Adapters

UF MUST NOT be used directly on external data without a domain adapter. Adapters define:

- valid input mappings for the domain,
- constraints on UF invocation,
- permissible and prohibited UF outputs,
- encryption and redaction rules via SES–Core (Section 15),
- governance-level interpretation rules (e.g., Section 16).

UF provides the structural engine; adapters enforce correctness and safety for specific domains.

1.5 Compliance Principle

Any deployment of UF MUST satisfy the following conditions:

- the UF Kernel MUST match this specification exactly,
- all UF outputs outside the Kernel boundary MUST be encrypted and redacted via SES–Core (Section 15),
- domain adapters MUST be used for all external interpretations,
- governance rules in Part V MUST be followed without exception.

UF–Spec v1.4.0 is therefore both a technical and regulatory document: it defines the system and the rules under which the system may lawfully be used.

2 Mathematical Overview

The Unified Framework (UF) is a deterministic structural transformation system. It operates on an input series

$$\mathcal{I} = \{(t_i, x_i)\}_{i=1}^n$$

and produces a chain of mathematically specified intermediates across Layers 0–4. Each layer applies a fixed operator to its input, and no layer modifies or adapts its parameters based on past data or outcomes. The resulting system is fully reproducible, auditable, and version-stable.

This section provides a high-level mathematical overview of the UF Kernel. All formal definitions, operators, and proofs appear in Section 11 (Formal Mathematics), which is the canonical source of truth for UF mathematics. Section 2 is conceptual and explanatory.

2.1 Sequential Input Domain

UF accepts a strictly ordered sequence:

$$(t_1, x_1), (t_2, x_2), \dots, (t_n, x_n),$$

with $t_i < t_{i+1}$ for all i . The system imposes no assumptions about the statistical nature of x_i ; the data may be noisy, adversarial, non-stationary, or irregular. UF evaluates structure—not distribution.

2.2 Layered Operator Model

UF applies five operators:

$$\mathcal{I} \xrightarrow{\mathcal{L}_0} SEV \xrightarrow{\mathcal{L}_1} \text{Gates} \xrightarrow{\mathcal{L}_2} \text{Interpretations} \xrightarrow{\mathcal{L}_3} \text{Resonance} \xrightarrow{\mathcal{L}_4} \text{Decision State Fields}.$$

Each operator is deterministic:

$$\mathcal{L}_k(x) = y \quad \text{for all valid inputs } x.$$

No operator:

- adapts its parameters,
- maintains hidden state,
- depends on past evaluations,
- uses learned weights or fitted coefficients.

2.3 Structural Evaluation Vectors (SEV)

L0 transforms the raw input sequence into a sequence of *Structural Evaluation Vectors*:

$$SEV(t_i) = (F_i, \Delta F_i, \sigma_i, \kappa_i, r_i, N_i).$$

These terms represent:

- F_i — the primary evaluation value (domain-specific),
- ΔF_i — local variation,
- σ_i — dispersion over a short window,
- κ_i — curvature or acceleration,
- r_i — relevance weight,
- N_i — negative-space indicator.

SEV is the fundamental structural representation from which all further UF operators derive their meaning. Full mathematical definitions are provided in Section 11.2.

2.4 Gate Segmentation (L1) Overview

L1 partitions the SEV sequence into "gates" based on structural deviation. A gate boundary occurs when:

$$D(t_i) > \tau_D,$$

where D is a deviation operator and τ_D is a global threshold. This segmentation creates localized structural regions that behave differently under subsequent UF layers.

2.5 Interpretive Layer (L2) Overview

L2 aggregates each gate into a small number of structural descriptors:

$$(w_k, CV_k, S_k, U_k, Reg_k).$$

These descriptors quantify:

- relative weight,
- contrast relative to a reference,
- structural score,
- uncertainty,
- structural regime.

The exact definitions appear in Section 11.3. L2 produces a compact characterization of each gate.

2.6 Resonance Engine (L3) Overview

L3 computes a normalized resonance value:

$$R(k) = f(w_k, CV_k, S_k, C_k, U_k),$$

with hysteresis and gating constraints to suppress unstable transitions.

The resonance sequence captures higher-order structural behavior across the gates. Peaks, troughs, and transitions in $R(k)$ reflect distinct structural phases in the underlying sequence.

2.7 Decision State Fields (L4) Overview

L4 computes the *Decision State Field* (DSF):

$$DSF_k = (D_k, M_k, R_k^*, U_k^*, P_k, B_k),$$

which describes the structural "state" of the system at gate k .

DSF is not a decision, forecast, or recommendation. It is a structural representation only.

2.8 Summary of Structural Flow

UF transforms a raw input series:

$$\mathcal{I} \rightarrow SEV \rightarrow \text{Gates} \rightarrow \text{Interpretations} \rightarrow \text{Resonance} \rightarrow DSF.$$

The system is purely structural, deterministic, and version-stable. External interpretation **MUST** occur through domain adapters (Part IV), and UF-SP **MUST** be protected by SES-Core (Section 15).

3 Structural Evaluation Vectors (SEV)

3.1 Purpose

L0 converts raw input streams into a mathematically well-posed Normalized Structural Field (NSF). It enforces signal integrity, derives structural quantities, and ensures deterministic behavior for all downstream layers.

3.2 Architecture Context

L0 is the first stage of the UF Kernel pipeline:

$$F(t) \rightarrow SEV(t) \rightarrow NSF$$

L1 gate formation depends entirely on correct L0 outputs.

3.3 Inputs

The input is a raw time-indexed field:

$$F : T \rightarrow \mathbb{R}^d.$$

3.4 Outputs

The output is the Normalized Structural Field:

$$NSF = \{SEV(t) \mid t \in T\}.$$

3.5 Input Health Verification

L0 **SHALL** reject the input if any of the following conditions fail:

- Timestamp monotonicity fails,
- $\|\Delta F(t)\| > \Delta_{\max}$,
- $\sigma(t) \notin [0, \sigma_{\max})$,
- $F(t) \notin D \subset \mathbb{R}^d$ (invalid domain).

These are normative constraints for SEV correctness.

3.6 Derived Structural Quantities

The following structural terms are derived:

$$\Delta F(t) = F(t) - F(t - \Delta t),$$

$$\sigma(t) = \frac{1}{W} \sum_{i=0}^{W-1} \|F(t - i\Delta t) - \bar{F}_W(t)\|^2,$$

$$\kappa(t) = \|F(t + \Delta t) - 2F(t) + F(t - \Delta t)\|.$$

These quantities represent local variation, dispersion, and curvature.

3.7 Negative-Space Operator

$$N(t) = \begin{cases} 1 & \sigma(t) < \sigma_{\min} \wedge \|\Delta F(t)\| < \delta_{\min} \wedge \kappa(t) < \kappa_{\min}, \\ 0 & \text{otherwise.} \end{cases}$$

Negative space identifies structurally “quiet” intervals.

3.8 State Embedding Vector (SEV)

The Structural Evaluation Vector is:

$$SEV(t) = (F(t), \Delta F(t), \sigma(t), \kappa(t), r(t), N(t)).$$

This is the fundamental structural representation for all downstream layers.

3.9 Perturbation Stability Constraint

SEV MUST satisfy:

$$\|SEV(F + \varepsilon) - SEV(F)\| \leq C\|\varepsilon\|.$$

This inequality ensures that SEV is stable under small perturbations, as required by Section 9 (Stability Metrics).

3.10 Role in Kernel Pipeline

L0 guarantees downstream determinism for:

- L1 gate formation,
- mosaic and schema projections,
- L2/L3 interpretive consistency,
- evolution and collapse diagnostics (Section 23).

L0 defines the structural foundation of UF.

4 Gate Segmentation and Mosaic Architecture

4.1 Purpose

L1 segments the NSF into gates and embeds them into one or more mosaic lattices for structural classification.

4.2 Architecture Context

L1 consumes L0 outputs:

$$NSF \rightarrow \{G_k, TVR_k, P_\ell(G_k), C_k\}$$

4.3 Gate Boundary Operator

$$D(t) = \alpha_1 \|\Delta F(t)\| + \alpha_2 \sigma(t) + \alpha_3 \kappa(t)$$

$$D(t) \geq \tau_D \Rightarrow \text{boundary}$$

4.4 Gate Definition

A gate $G_k = [t_a, t_b]$ satisfies:

$$\forall t \in (t_a, t_b) : D(t) < \tau_D$$

4.5 Time–Volume–Relevance Vector

$$TVR_k = (T_k, V_k, R_k)$$

$$T_k = t_b - t_a$$

$$V_k = \int_{t_a}^{t_b} (\beta_1 \|\Delta F\| + \beta_2 \sigma + \beta_3 \kappa) dt$$

$$R_k = \int_{t_a}^{t_b} r(t) dt$$

4.6 Mosaic Projection Operator

$$P_\ell(G_k) = \left(\left\lfloor \frac{T_k}{h_1^{(\ell)}} \right\rfloor, \left\lfloor \frac{V_k}{h_2^{(\ell)}} \right\rfloor, \left\lfloor \frac{R_k}{h_3^{(\ell)}} \right\rfloor \right)$$

4.7 Mosaic Divergence

$$C_k = |\{P_1(G_k), \dots, P_L(G_k)\}|$$

4.8 Gate Drift

$$\delta_g(G_k) = \|\mu_k - \mu_{k-1}\|$$

4.9 Role in Kernel Pipeline

L1 determines the structural segmentation and quantization that all downstream interpretive, resonance, and decision mechanisms depend on.

5 Interpretive Structural Metrics

5.1 Purpose

L2 transforms gate-level descriptors produced by L1 into structurally meaningful interpretive metrics. These metrics quantify intensity, contrast, stability, structural uncertainty, regime classification, and anomaly detection for downstream resonance and decision dynamics.

5.2 Architecture Context

L2 consumes L1 outputs:

$$\{G_k, TVR_k, P_\ell(G_k), C_k, \delta_g(G_k), N(G_k)\}$$

and produces:

$$ISF_k = (w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

5.3 Inputs

- Gate structure: $G_k = [t_a, t_b]$
- Time–Volume–Relevance vector: $TVR_k = (T_k, V_k, R_k)$
- Mosaic projections: $P_\ell(G_k)$
- Divergence: C_k
- Gate drift: $\delta_g(G_k)$
- Negative-space gate flag: $N(G_k)$

5.4 Outputs

Interpretive Structural Field (ISF):

$$ISF_k = (w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

5.5 Interpretive Weight

$$w_k \in [0, 1]$$

represents normalized structural intensity derived from average first difference, variance, and curvature within G_k .

5.6 Contrast Vector

$$CV_k = TVR_k - \mu_k$$

where μ_k is the local structural baseline across neighboring gates.

5.7 Structural Score

$$S_k = \gamma_1 w_k + \gamma_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \gamma_3 \frac{1}{1 + C_k}$$

5.8 Regime Classification

Given structural density $\chi_k = V_k/T_k$ and normalized contrast $\psi_k = \|CV_k\|/\|CV\|_{\max}$:

- Stable: $\chi_k < \chi_{\min}$ and $\psi_k < \psi_{\min}$
- Transitional: $\chi_{\min} \leq \chi_k \leq \chi_{\max}$
- Volatile: $\chi_k > \chi_{\max}$
- Degenerate: $\psi_k > \psi_{\max}$

5.9 Uncertainty

$$U_k = \lambda_1 \frac{C_k - 1}{L - 1} + \lambda_2 \frac{\delta_g(G_k)}{\delta_{\max}} + \lambda_3 N(G_k)$$

5.10 Interpretive Anomaly Suppression

$$IAS_k = 1 \iff U_k > U_{\max}$$

5.11 Role in Kernel Pipeline

L2 provides the structural metrics that determine resonance alignment, anomaly suppression, and decision dynamics.

6 Resonance Engine

6.1 Purpose

L3 aggregates interpretive metrics into a single scalar resonance measure $R(k)$. Resonance quantifies structural alignment, stability, and interpretive coherence for each gate.

6.2 Architecture Context

L3 consumes L2 outputs:

$$(w_k, CV_k, S_k, Reg_k, U_k, IAS_k)$$

and produces:

$$URF_k = g_k \cdot R(k)$$

6.3 Resonance Vector

$$R_k^{vec} = \left(w_k, \frac{\|CV_k\|}{\|CV\|_{\max}}, S_k, \frac{1}{1 + C_k}, 1 - U_k \right)$$

6.4 Resonance Functional

$$R(k) = \frac{1}{Z} \left(\lambda_1 w_k + \lambda_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \lambda_3 S_k + \lambda_4 \frac{1}{1 + C_k} + \lambda_5 (1 - U_k) \right)$$

with

$$Z = \sum_i \lambda_i.$$

6.5 Resonance Hysteresis

$$Hyst_k = 1 \iff |R(k) - R(k-1)| > h_{\max}$$

6.6 Contextual Gating

$$g_k = 1 \iff (U_k \leq U_{\max}) \wedge (IAS_k = 0) \wedge (Hyst_k = 0)$$

6.7 Unified Resonance Field

$$URF_k = g_k \cdot R(k)$$

6.8 Role in Kernel Pipeline

L3 provides the scalar resonance signal that informs decision dynamics (L4), ensuring structural consistency and stability across interpretive transitions.

7 Decision Dynamics

7.1 Purpose

L4 transforms resonance signals from L3 into directional, momentum, reversal, uncertainty-adjusted, and breathing-aware components that form the Decision Surface Vector (DSF). The DSF is the sole interface from the kernel to L5 domain adapters.

7.2 Architecture Context

L4 consumes L3 outputs:

$$URF_k = g_k \cdot R(k)$$

and produces:

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$$

7.3 Directional Signal

Let $\Delta R(k) = R(k) - R(k-1)$. Define:

$$D_k = \begin{cases} +1 & \Delta R(k) > \epsilon_D \\ 0 & |\Delta R(k)| \leq \epsilon_D \\ -1 & \Delta R(k) < -\epsilon_D \end{cases}$$

7.4 Momentum and Reversal

$$M_k = R(k) - 2R(k-1) + R(k-2)$$

$$R_k = 1 \iff D_k \cdot D_{k-1} < 0$$

7.5 Adjusted Uncertainty

$$U_k^* = U_k + \eta_H Hyst_k + \eta_{IAS} IAS_k$$

7.6 Pressure Metric

$$P_k = |D_k - D_{k-1}|$$

7.7 Breathing Dynamics

$$B_k = B_{k-1} + \xi(1 - U_k^*)\Delta R(k) - \chi U_k^*$$

with clamping:

$$B_k = \max(B_{\min}, \min(B_{\max}, B_k))$$

7.8 Decision Surface Vector

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$$

7.9 Role in Kernel Pipeline

L4 determines structural direction, stability, interpretive load, and momentum supplied to L5, while preserving kernel invariants.

Part II — Global Properties

8 Invariants

The Unified Framework (UF) is governed by a collection of invariants— mathematical properties that **MUST** hold for every valid UF evaluation, regardless of domain, input series, or adapter layer. These invariants ensure that UF is deterministic, reproducible, and structurally stable across implementations and versions. Invariants apply globally across Layers 0–4 and define the fundamental behavior of the system.

This section provides a conceptual overview of UF invariants. Formal mathematical expressions appear in Section 11 (Formal Mathematics), which is the canonical source of truth.

8.1 Deterministic Operator Invariance

Each UF operator $\mathcal{L}_k$ is a deterministic map:

$$\mathcal{L}_k : X \rightarrow Y.$$

For fixed inputs, the output of $\mathcal{L}_k$ **MUST** be bit-identical across:

- independent implementations,
- different hardware platforms,
- repeated evaluations,
- reordered or parallel execution environments.

This ensures that UF is not sensitive to machine state, thread scheduling, or non-deterministic execution environments.

8.2 Invariance to Null Structural Transforms

If the input series is transformed by any structure-preserving function T such that:

$$T(x_i) = x_i \quad \text{or} \quad T(x_i) = x_i + c \quad \text{for constant } c,$$

then UF outputs **MUST** satisfy:

$$UF(T(\mathcal{I})) = UF(\mathcal{I}),$$

up to the SEV components explicitly sensitive to such transformations (e.g. ΔF when constant offsets preserve differences).

UF **MUST** preserve structural equivalence classes. Only changes in structure— not uniform shifts or domain-allowed equivalences—may affect UF outputs.

8.3 Gate Boundary Invariance

Gate boundaries MUST satisfy:

$$D(t_i) > \tau_D \quad \Rightarrow \quad \text{boundary exists independently of evaluation context.}$$

This means:

- windowed or batch evaluations MUST produce identical boundaries,
- different domain adapters MUST NOT alter L1 segmentation,
- L1 MUST NOT depend on version-specific heuristics,
- parallel execution MUST NOT change boundary placement.

Gate segmentation is a globally stable structural operation.

8.4 Interpretive Layer Invariance

For any gate G_k , the interpretation tuple:

$$(w_k, CV_k, S_k, U_k, Reg_k)$$

MUST satisfy the following invariants:

- Regime classification MUST be invariant to evaluation ordering,
- w_k MUST be a deterministic function of SEV and gate structure,
- CV_k MUST be bounded and scale-normalized,
- S_k MUST be independent of domain semantics,
- U_k MUST reflect structural uncertainty only.

These invariants ensure that L2 conveys structural meaning independent of domain assumptions or numerical magnitudes.

8.5 Resonance Invariance

The resonance function $R(k)$ MUST satisfy:

$$R(k) = f(w_k, CV_k, S_k, C_k, U_k),$$

with a normalization constant Z fixed in UF-Spec v1.4.0. The following invariants MUST hold:

- resonance MUST be invariant to absolute scale of F ,

- resonance MUST be invariant to uniform translation of F ,
- resonance MUST reflect structural changes only (not magnitude),
- hysteresis MUST be applied deterministically.

Resonance encodes second-order structure and MUST NOT vary across implementations.

8.6 DSF Invariance

The Decision State Field:

$$DSF_k = (D_k, M_k, R_k^*, U_k^*, P_k, B_k)$$

MUST satisfy:

- D_k MUST reflect directional resonance deltas only,
- M_k MUST reflect momentum invariants defined in Section 11,
- R_k^* MUST reflect resonance reversal invariants,
- U_k^* MUST be bounded to $[0, 1]$,
- P_k and B_k MUST reflect structural pressure dynamics.

DSF MUST NOT encode domain semantics or predictions.

8.7 Version Invariance

UF evaluations MUST satisfy:

$$UF_v(x) = UF_v(x) \quad \forall x,$$

and MUST NOT silently change across versions. Any modification to UF operators requires:

- a new UF version identifier,
- updated validation suite,
- updated SES-Core version (if UF-SP format changes),
- updated governance rules.

8.8 Summary of Invariants

UF invariants provide global guarantees:

- deterministic operator behavior,
- structural input equivalence,
- segmentation invariance,
- interpretive invariance,
- resonance invariance,
- DSF invariance,
- version isolation.

These properties define UF as a version-stable, domain-independent, structurally deterministic evaluation engine.

9 Stability Metrics

Stability metrics quantify how the Unified Framework (UF) responds to small perturbations of its input series. These metrics do not measure predictive accuracy, statistical fit, or model confidence. Instead, they measure the *structural coherence* of the system across Layers 0–4.

The canonical stability metrics of UF are:

- the Structural Stability Score $S(UF)$,
- the Robustness Score $R(UF)$,
- Gate Stability Score (GSS),
- Directional Stability Score (DSS),
- DSF Stability Score.

All stability metrics are domain-independent and MUST be computed in accordance with Section 13 (Validation) and Section 11 (Formal Mathematics).

9.1 Perturbation Model

Stability is defined by comparing UF evaluations on:

$$\mathcal{I} \quad \text{and} \quad \mathcal{I}_\epsilon,$$

where $\mathcal{I}_\epsilon$ is obtained by applying a multiplicative noise perturbation:

$$x'_i = x_i(1 + \epsilon_i), \quad \epsilon_i \in [-\sigma_{pct}, +\sigma_{pct}].$$

Valid perturbation levels are:

$$\sigma_{pct} \in \{0.1\%, 0.2\%, 0.5\%, 1.0\%, 2.0\%\}.$$

The perturbation operator MUST preserve:

- input ordering,
- positivity of valid input fields (where applicable),
- consistency of UF input format.

9.2 Gate Stability Score (GSS)

GSS measures whether perturbations change the segmentation produced by L1. Formally:

$$GSS = 1 - \frac{\#(\text{changed gate boundaries})}{\#(\text{boundaries})}.$$

Thus:

- $GSS = 1$ means gate boundaries are identical,
- $GSS = 0$ means all boundaries differ,
- intermediate values reflect partial consistency.

Gate segmentation MUST be invariant under small perturbations for the system to be considered structurally stable.

9.3 Directional Stability Score (DSS)

DSS measures the sensitivity of the directional signal:

$$D_k = \begin{cases} +1, & \Delta R(k) > \epsilon_D, \\ 0, & |\Delta R(k)| \leq \epsilon_D, \\ -1, & \Delta R(k) < -\epsilon_D. \end{cases}$$

DSS is defined as:

$$DSS = 1 - \frac{\#(D_k \neq D'_k)}{\#(\text{gates})}.$$

It measures whether small perturbations flip the directional classification.

9.4 DSF Stability Score

DSF Stability Score measures the consistency of the full Decision State Field across perturbations:

$$DSF_Stability = 1 - \frac{\#(\text{DSF mismatch coordinates})}{\#(\text{total DSF coordinates})}.$$

This metric is the most sensitive stability indicator because DSF combines directional, momentum, pressure, reversal, and breathing components.

9.5 Composite Structural Stability Score $S(UF)$

$S(UF)$ summarizes overall structural stability by combining:

- Gate Stability Score (GSS),
- Directional Stability Score (DSS),
- DSF Stability Score.

A normative composite function is defined in Section 11.6. Properties:

- $S(UF) \in [0, 1]$,
- $S(UF) = 1$ means complete structural invariance,
- $S(UF) = 0$ means total instability.

$S(UF)$ is domain-independent and MUST NOT encode domain semantics.

9.6 Robustness Score $R(UF)$

$R(UF)$ measures how structure responds to *systematic* perturbation magnitudes:

$R(UF) = \text{aggregate response over } \sigma_{pct} \in \{0.1\%, 0.2\%, 0.5\%, 1.0\%, 2.0\%\}.$

$R(UF)$ MUST satisfy:

- monotonic degradation under increasing perturbation,
- boundedness in $[0, 1]$,
- deterministic reproducibility.

$R(UF)$ is not a risk or uncertainty measure; it is a structural robustness indicator.

9.7 Relationship Between Stability Metrics and Invariants

UF invariants (Section 8) define global constraints that **MUST** hold for every evaluation. Stability metrics measure *how strongly* these invariants hold under perturbation.

$S(UF) = 1 \Rightarrow$ all invariants remain unchanged by perturbation.

$R(UF) = 0 \Rightarrow$ invariants collapse under perturbation.

Stability metrics quantify structural resilience, not prediction quality.

9.8 Summary of Stability Metrics

UF Stability Metrics:

- characterize structural sensitivity,
- measure invariance under controlled perturbations,
- support domain-independent evaluation,
- enable safe externalization of UF outputs via SES-Core,
- provide governance-layer signals for adapters.

Stability metrics define the structural “strength” of an input series under UF evaluation.

10 Hardening Model (Kernel Safety)

Clarification: All SES references in Section 10 SHALL be interpreted in accordance with Section 15 (SES-Core v2.0). The Hardening Model operates strictly *within* the UF Kernel boundary and SHALL NOT be construed as requiring exposure of any UF Structural Payload (UF-SP) outside the Kernel. Whenever this section refers to “SES behavior” or “SES evolution,” such phrases SHALL be interpreted as referring to the observable, redacted SES Summary fields described in Section 15, not to internal SES state or ciphertext structures. Hardening acts only on Kernel-internal state and its own diagnostic signals; SES-Core governs UF-SP externalization.

Purpose: Define the Unified Framework (UF) safety controls that prevent structural instability, ensure bounded behavior across all kernel layers (L0–L4), enforce invariants, and direct SafeMode activation and recovery. Hardening ensures that the UF never produces runaway states, invalid outputs, or unbounded schema/mosaic dynamics.

10.1 Hardening Overview

Hardening consists of three components:

- **Hardening Trigger Conditions (HTCs)** — When the UF must intervene.
- **Hardening Actions (HAs)** — Exact responses taken when a trigger fires.
- **Hardening Recovery Rules (HRRs)** — How the UF returns to normal mode.

Hardening applies continuously in L0–L4 and is referenced by all Section 13 protocols.

10.2 Hardening Trigger Conditions (HTCs)

An HTC fires when ANY safety-critical invariant is violated or crosses its warning threshold.

HTC-1: Mosaic Divergence Violation

$$C_k > C_{\max}$$

HTC-2: Uncertainty Violation

$$U_k > U_{\max}$$

HTC-3: Breathing Range Violation

$$B_k < B_{\min} \quad \text{or} \quad B_k > B_{\max}$$

HTC-4: Schema Growth Violation

$$S_{\text{count}} > S_{\max}$$

HTC-5: Schema Entropy Violation

$$H_s > H_{\max}$$

HTC-6: Stability Collapse

$$S(\text{UF}) < S_{\min}$$

HTC-7: DSF Instability

$$\|\text{DSF}_k - \text{DSF}_{k-1}\| > \varepsilon_{\text{DSF}}$$

in the absence of a valid resonance transition ($|\Delta R(k)| \leq \varepsilon_D$).

HTC-8: Drift Violation

$$\text{Drift}(T) > D_{\max}$$

HTC-9: Resonance Blowup

$$R(k) \notin [0, 1]$$

(due to corruption or failed gating).

HTC-10: SES Decryption Failure

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 0$$

10.3 Hardening Actions (HAs)

When ANY HTC fires, the UF applies one or more of these deterministic actions.

HA-1: Gate Suppression

The current gate G_k is suppressed:

$$g_k = 0$$

This prevents invalid resonance or decision propagation.

HA-2: DSF Neutralization

The DSF is replaced by a neutral vector:

$$\text{DSF}_k = (0, 0, 0, 1, C_k, 0, B_{k-1})$$

This stabilizes reasoning to a “hold” position.

HA-3: Uncertainty Spike

$$U_k \leftarrow 1$$

This prevents L4 from making high-confidence transitions.

HA-4: Breathing Contraction

$$B_k \leftarrow \max(B_{\min}, B_k - \beta_B)$$

HA-5: Schema Freeze

No new schemas may form:

$$\Delta S_{\text{count}} = 0$$

HA-6: Schema Merge

If schemas are proliferating:

$$\text{merge all } C_i \text{ with } \|C_i - C_j\| < \tau_{\text{merge}}$$

HA-7: Mosaic Compression

Force mosaic projections toward consensus:

$$C_k \leftarrow 1 + \min(C_k - 1, \gamma_C)$$

HA-8: SES Failsafe Mode

Disable encryption for this cycle and emit warning:

$$\text{SES_status} = \text{"failsafe"}$$

10.4 SafeMode

SafeMode is a global state in which:

- All decision dynamics freeze.
- DSF is replaced by a neutral hold vector.
- Schema formation is suspended.
- Mosaic projections are clamped.
- L3 resonance gating g_k is forced to 0.
- Hardening remains active until recovery rules are met.

Formal condition:

$$\text{SafeMode} = 1 \iff S(\text{UF}) < S_{\min}$$

In SafeMode, the system may still interpret input, but cannot:

- escalate resonance
- update schemas
- expand breathing
- produce directional decisions

10.5 Hardening Recovery Rules (HRRs)

To exit SafeMode or normal hardening:

HRR-1: Stability Recovery

$$S(\text{UF}) \geq S_{\min}$$

HRR-2: Mosaic Recovery

$$C_k \leq C_{\max}$$

HRR-3: Breathing Recovery

$$B_{\min} \leq B_k \leq B_{\max}$$

HRR-4: Schema Recovery

$$S_{\text{count}} \leq S_{\max}$$

HRR-5: SES Recovery

$$\text{SES_status} = \text{"ok"}$$

When ALL HRRs are satisfied:

$$\text{SafeMode} \leftarrow 0$$

and all kernel layers resume normal operation.

10.6 Traceability

- Section 8 — Invariants
- Section 9 — Stability Metrics
- Section 13 — Validation Requirements (all tests reference these HTCs/HAs/HRRs)
- Section 15 — SES compatibility and error handling
- Section 19 — Compliance and Reproducibility
- Section 22 — Governance enforcement

10.7 Summary

Section 10 defines the precise safety boundaries for UF operation. All kernel layers (L0–L4), validation protocols, and SES integration rely on these controls. A UF instance that violates any HTC without triggering a corresponding Hardening Action or SafeMode is non-compliant with UF-Spec v1.4.0.

Section 11 — Formal Mathematical Framework

Purpose

This section defines the mathematical operators, constraints, and invariants that govern the Unified Framework. All other sections must be consistent with these definitions.

State Embedding Vector

$$SEV(t) = (F(t), \Delta F(t), \sigma(t), \kappa(t), r(t), N(t))$$

Perturbation stability:

$$\|SEV(F + \varepsilon) - SEV(F)\| \leq C_{SEV} \|\varepsilon\|.$$

Gate Boundary Operator

$$D(t) = \alpha_1 \|\Delta F(t)\| + \alpha_2 \sigma(t) + \alpha_3 \kappa(t)$$

Gate opens when:

$$D(t) < \tau_D.$$

Mosaic Projection

$$P_\ell(G_k) = \left(\left\lfloor \frac{T_k}{h_1^{(\ell)}} \right\rfloor, \left\lfloor \frac{V_k}{h_2^{(\ell)}} \right\rfloor, \left\lfloor \frac{R_k}{h_3^{(\ell)}} \right\rfloor \right)$$

Divergence:

$$C_k = |\{P_1(G_k), \dots, P_L(G_k)\}|.$$

Contrast Vector

$$CV_k = TVR_k - \mu_k.$$

Resonance Functional

$$R(k) = \frac{1}{Z} \left(\lambda_1 w_k + \lambda_2 \frac{\|CV_k\|}{\|CV\|_{\max}} + \lambda_3 S_k + \lambda_4 \frac{1}{1 + C_k} + \lambda_5 (1 - U_k) \right).$$

Hysteresis

$$Hyst_k = 1 \iff |R(k) - R(k-1)| > h_{\max}.$$

Decision Surface Formalism

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k).$$

Global Stability Functional

$$S(UF) = \lambda_c(1-Coh) + \lambda_h(1-H_s) + \lambda_b(1-Var(B)) + \lambda_u(1-\bar{U}) + \lambda_d(1-Drift(T)).$$

Robustness Index

$$R(UF) = 1 - \frac{1}{K} \sum_{i=1}^K |S(UF) - S(UF^{(i)})|.$$

Invariants

$$0 \leq R(k) \leq 1, \quad 0 \leq U_k \leq 1, \quad C_k \leq C_{\max}, \quad H_s \leq H_{\max}, \quad B_{\min} \leq B_k \leq B_{\max}.$$

SafeMode triggers when:

$$S(UF) < S_{\min}.$$

Part III — Validation

12 Academic Validation Layer

The purpose of the Academic Validation Layer is to establish the theoretical justification for the procedures defined in Section 13. Validation in UF is not an empirical calibration process, nor is it an attempt to align UF with domain-specific expectations. Instead, validation demonstrates that the Unified Framework behaves consistently with its mathematical definition and satisfies the invariants and stability properties defined in Part II.

UF validation is therefore *structural*: it evaluates whether UF correctly preserves deterministic, reproducible behavior across perturbation and configuration tests, and whether UF produces coherent Decision State Fields (DSFs) under minimal transformations of the input series.

12.1 Purpose of Validation

The goals of UF validation are:

- to demonstrate that UF satisfies the invariants in Section 8,
- to confirm that perturbation stability behaves as defined in Section 9,
- to expose collapse modes that indicate boundary conditions of UF,
- to ensure reproducibility of structural evaluations,
- to show that UF's deterministic pipeline behaves independently of domain semantics,
- to ensure that UF outputs remain well-formed under noise, adversarial drift, and rescaling,

- to provide a foundation for domain adapters that rely on SES Summary fields.

UF validation does not attempt to guarantee domain performance. It guarantees structural correctness.

12.2 Validation and the UF Kernel Boundary

All UF validation occurs *inside* the Kernel boundary. This distinction is essential because UF internal structures (SEV, gates, L2/L3 intermediates, DSF, and Section 23 hardening diagnostics) **MUST NOT** be exposed outside the Kernel except through SES-Core encryption and redaction (Section 15).

During validation:

- UF-SP is fully visible internally,
- perturbation operators are applied directly to input series,
- raw UF structures are compared and analyzed,
- no UF-SP leaves the Kernel boundary.

Thus, validation does not violate SES-Core policy.

12.3 Perturbation as a Structural Probe

Validation relies on perturbing the input series and evaluating the effect on structural outputs. Perturbation acts as a *probe* of UF invariants. If the system responds coherently under perturbation, it is stable. If minor perturbations collapse the structural representation, the system is unstable.

UF does not randomize its internals; therefore, perturbation reveals intrinsic properties of the input structure, not model uncertainty or predictive error.

12.4 Collapse Modes

Collapse occurs when one or more invariants fail to hold under perturbation. Examples include:

- gate boundaries changing unpredictably,
- instability in directional classification D_k ,
- incoherent Decision State Fields,
- inconsistency in resonance reversal detection,
- violation of boundedness constraints (Section 11),
- L2 or L3 values becoming undefined or non-finite.

Collapse is not an error of implementation; it is a *property of the input structure*. UF **MUST** detect it and propagate collapse indicators to the SES Summary fields consumed by domain adapters.

12.5 Reproducibility Requirements

Validation MUST demonstrate reproducibility in two dimensions:

- **Run-to-run reproducibility:** repeated evaluations on the same input series MUST yield identical UF outputs.
- **Implementation-level reproducibility:** independent implementations MUST produce bit-identical evaluation results (within allowed floating-point tolerances defined in Section 11).

If reproducibility fails, the implementation is non-conformant with UF-Spec v1.4.0.

12.6 Validation as the Academic Foundation of Adapters

Domain adapters (Part IV) rely on SES Summary fields, which are structurally derived from validation concepts:

- stability metrics (Section 9),
- collapse indicators (Section 23),
- SafeMode signals,
- deterministic behavior guarantees (Section 8).

Validation therefore provides the academic foundation for all domain-specific interpretation rules. Adapters MUST NOT reinterpret UF internals beyond what validation demonstrates is structurally meaningful.

12.7 Summary of Academic Validation

UF validation:

- verifies invariants,
- quantifies stability,
- detects collapse,
- enforces reproducibility,
- constrains domain adapters,
- ensures SES-Core redaction consistency,
- provides the mathematical basis for safe external interpretation.

Section 12 defines *why* validation is required. Section 13 defines *how* validation is performed.

13 Validation & Benchmarking Suite

13.0 Purpose

This section defines deterministic validation procedures ensuring Unified Framework (UF) structural correctness, robustness, bounded schema growth, mosaic consistency, SES compatibility, and shock recovery.

13.1 Validation Concepts

13.1.1 Global Stability Functional

The stability functional is defined as:

$$S(\text{UF}) = \lambda_c(1 - \text{Coh}) + \lambda_h(1 - H_s) + \lambda_b(1 - \text{Var}(B)) + \lambda_u(1 - \bar{U}) + \lambda_d(1 - \text{Drift}(T)).$$

13.1.2 Robustness Index

For perturbation trials:

$$R(\text{UF}) = 1 - \frac{1}{K} \sum_{i=1}^K |S(\text{UF}) - S(\text{UF}^{(i)})|.$$

13.1.3 Structural Validation Domains

Validation spans:

- low-structure fields,
- high-structure fields,
- adversarial structural fields.

13.2 Core Validation Requirements

UF MUST satisfy:

- **DSF bounded variation:**

$$\|\text{DSF}_k - \text{DSF}_{k-1}\| \leq \varepsilon_{\text{DSF}} \quad \text{unless } |\Delta R(k)| > \varepsilon_D.$$

- **Entropy bound:** $H_s \leq H_{\max}$.
- **Mosaic divergence bound:** $C_k \leq C_{\max}$.
- **Cross-layer consistency:** $\text{XLC}(k) = 1$.
- **Breathing stability:** $\text{Var}(B) \leq \varepsilon_B$.

13.3 Noise & Perturbation Robustness

Under bounded disturbance:

$$F_\varepsilon(t) = F_0(t) + \varepsilon u(t),$$

UF MUST satisfy:

$$\Delta S(\varepsilon) \leq \kappa_S \varepsilon, \quad \Delta U(\varepsilon) \leq \kappa_U \varepsilon, \quad \text{Var}(B_\varepsilon) \leq \text{Var}(B_0) + \kappa_B \varepsilon.$$

13.4 Adversarial Structured Input Tests

Adversarial classes maximize drift, curvature, divergence, and uncertainty. UF MUST satisfy:

$$\Delta S_X \leq \kappa_{\text{ADV}} \varepsilon_X, \quad U_X \leq U_{\max}, \quad C_k(X) \leq C_{\max}, \quad 0 \leq R(k) \leq 1.$$

13.5 Parameter Perturbation Tests

$$\Delta S_\Theta = |S(\text{UF}_\Theta) - S(\text{UF}_{\Theta'})| \leq \kappa_{S\Theta} \varepsilon_\Theta.$$

13.6 Memory Load & Schema Stress Tests

UF MUST maintain:

$$S_{\text{count}} \leq S_{\max}, \quad H_s \leq H_{\max}, \quad \text{Drift}(T) \leq D_{\max}.$$

13.7 Multi-Lattice Conflict Tests

UF MUST satisfy:

$$C_k \leq C_{\max}, \quad U_k \leq U_{\max}, \quad S(\text{UF}, k) \geq S_{\min}.$$

13.8 Breathing–Self-Organization Synchronization

UF MUST satisfy:

$$A_B(k) \mu_s(k) \leq \beta_{\max}, \quad S(\text{UF}, k) \geq S_{\min}.$$

13.9 SES Evolution Tests

UF MUST satisfy:

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 1, \quad \Delta K_k \leq \kappa_K \varepsilon_\Theta.$$

13.10 Cross-Domain Transfer Tests

UF MUST satisfy:

$$\Delta S_X \leq \kappa_{SD} d_X, \quad \text{Var}(B_X) \leq \text{Var}(B_0) + \kappa_{BD} d_X.$$

13.11 Global Stability Shock Tests

Shock scenarios MUST preserve boundedness:

$$S(\text{UF}_{\text{shock}}) \geq 0, \quad C_k \leq C_{\max}, \quad B_k \in [B_{\min}, B_{\max}].$$

13.12 Validation Summary

UF is validated only if **all** structural, robustness, SES, mosaic, schema, breathing, and shock criteria are simultaneously satisfied.

Clarification: All UF outputs referenced in Section 13 are evaluated strictly *inside* the UF Kernel boundary. No UF Structural Payload (UF-SP) is ever exposed externally during validation. Internal access to UF-SP for computing stability metrics, perturbation responses, and collapse indicators is fully compliant with SES-Core rules, which govern only UF-SP handling *outside* the Kernel boundary.

13.x Executable Validation Protocols

13.0.0 Global Protocol Standards

13.0.0 Global Protocol Standards

Purpose. Establish universal preconditions, structural requirements, execution flow, and deterministic evaluation criteria for all validation protocols defined in Section 13.

Purpose

Describe the purpose of this validation test.

Preconditions

List all system and environmental requirements.

Required Data

Specify datasets or synthetic data generation rules.

Execution Steps

1. Step 1
2. Step 2
3. Step 3

Expected Results

Describe the expected system behavior.

Pass/Fail Criteria

Use deterministic inequalities (no probability).

Traceability

- Kernel Invariants Referenced:
- Hardening Conditions:
- SES Requirements:

13.3.1 Noise and Perturbation Robustness

13.3.1 Noise and Perturbation Robustness

Goal: Verify that the UF engine satisfies the deterministic robustness inequalities defined in Section 13.3 when exposed to bounded additive noise.

Preconditions

- Python 3.9+ is installed.
- Folder structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Baseline structural field exists at:

```
inputs/baseline.json
```

- Noise generator script exists at:

```
scripts/generate_noise.py
```

- config.yaml from the UF-Spec v1.4.0 baseline is present.

Required Data

- Baseline input file:

`inputs/baseline.json`

- Noise levels (deterministic sequence):

$$\varepsilon \in \{0.01, 0.05, 0.10, 0.20, 0.30\}$$

Setup

Open a terminal and run these commands from inside `UF-Engine/`:

```
python scripts/generate_noise.py --epsilon 0.01 --input inputs/baseline.json --output inputs/test_13-1_noise_0.01.json
python scripts/generate_noise.py --epsilon 0.05 --input inputs/baseline.json --output inputs/test_13-1_noise_0.05.json
python scripts/generate_noise.py --epsilon 0.10 --input inputs/baseline.json --output inputs/test_13-1_noise_0.10.json
python scripts/generate_noise.py --epsilon 0.20 --input inputs/baseline.json --output inputs/test_13-1_noise_0.20.json
python scripts/generate_noise.py --epsilon 0.30 --input inputs/baseline.json --output inputs/test_13-1_noise_0.30.json
```

This produces five deterministic noisy input files.

Execution Steps

1. Run the UF engine on the baseline file:

```
python uf_engine.py --config config.yaml \
--input inputs/baseline.json \
--output outputs/baseline_result.json \
--log logs/baseline.log
```

2. Run the UF engine on each noisy input file:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.01.json \
--output outputs/test_13-1_noise_0.01_result.json \
--log logs/test_13-1_noise_0.01.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.05.json \
--output outputs/test_13-1_noise_0.05_result.json \
--log logs/test_13-1_noise_0.05.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.10.json \
--output outputs/test_13-1_noise_0.10_result.json \
--log logs/test_13-1_noise_0.10.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.20.json \
--output outputs/test_13-1_noise_0.20_result.json \
--log logs/test_13-1_noise_0.20.log
```

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-1_noise_0.30.json \
--output outputs/test_13-1_noise_0.30_result.json \
--log logs/test_13-1_noise_0.30.log
```

3. For each output file in `outputs/`, record:

- `"delta_S"`
- `"delta_U"`
- `"delta_Var_B"`
- `"invariant_violations"`

Expected Results

For each noise level:

- `"invariant_violations"` MUST equal:

 `[]`
- `"delta_S"`, `"delta_U"`, and `"delta_Var_B"` must be:
 - non-negative,
 - increasing monotonically with ε ,
 - satisfying UF-Spec deterministic bounds.

Pass / Fail Criteria

PASS if all noise levels satisfy:

- `"invariant_violations"` is empty.
- `"delta_S"` $\leq \kappa_S \varepsilon$
- `"delta_U"` $\leq \kappa_U \varepsilon$
- `"delta_Var_B"` $\leq \kappa_B \varepsilon$

FAIL if any noise level shows:

- non-empty invariant violations,
- non-monotonic deviations,
- any breach of the deterministic bounds.

Traceability

- Stability functional $S(\text{UF})$ (Section 13.1)
- Robustness bounds (Section 13.3)
- Hardening & SafeMode conditions (Section 10)
- L0–L4 operational equations (Part I)

13.4.1 Adversarial Structured Input Tests

13.4.1 Adversarial Structured Input Tests

Goal: Verify that the UF engine maintains structural invariants and bounded behavior when exposed to adversarially constructed input fields designed to maximize drift, curvature, variance, and mosaic divergence.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Baseline field exists:

```
inputs/baseline.json
```

- Adversarial generator script:

```
scripts/generate_adversarial.py
```

Required Data

Adversarial classes A–E must be generated:

- Class A: maximize drift
- Class B: maximize curvature
- Class C: maximize variance
- Class D: maximize mosaic divergence
- Class E: combined adversarial pattern

Generated filenames follow:

```
inputs/test_13-2_adv_A.json  
inputs/test_13-2_adv_B.json  
inputs/test_13-2_adv_C.json  
inputs/test_13-2_adv_D.json  
inputs/test_13-2_adv_E.json
```

Setup

Run from inside UF-Engine/:

```
python scripts/generate_adversarial.py --mode A --output inputs/test_13-2_adv_A.json
python scripts/generate_adversarial.py --mode B --output inputs/test_13-2_adv_B.json
python scripts/generate_adversarial.py --mode C --output inputs/test_13-2_adv_C.json
python scripts/generate_adversarial.py --mode D --output inputs/test_13-2_adv_D.json
python scripts/generate_adversarial.py --mode E --output inputs/test_13-2_adv_E.json
```

Execution Steps

1. Run the UF engine on each adversarial input:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_A.json \
--output outputs/test_13-2_adv_A_result.json \
--log logs/test_13-2_adv_A.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_B.json \
--output outputs/test_13-2_adv_B_result.json \
--log logs/test_13-2_adv_B.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_C.json \
--output outputs/test_13-2_adv_C_result.json \
--log logs/test_13-2_adv_C.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_D.json \
--output outputs/test_13-2_adv_D_result.json \
--log logs/test_13-2_adv_D.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-2_adv_E.json \
--output outputs/test_13-2_adv_E_result.json \
--log logs/test_13-2_adv_E.log
```

2. Extract from each output:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For every adversarial class:

- No crashes or halts.
- "invariant_violations" must be:

[]

- Breathing values stay within allowed range.
- Mosaic divergence remains:

$$C_k \leq C_{\max}$$

- Uncertainty remains:

$$U_k \leq U_{\max}$$

- DSF components remain finite (no NaN/INF).

Pass / Fail Criteria

PASS if:

- No invariant violations
- Drift, mosaic, and uncertainty remain within bounds
- No SafeMode misfires
- DSF does not blow up

FAIL if:

- Any invariant violation
- Out-of-bounds breathing, divergence, uncertainty
- NaN or INF in DSF
- Unexpected SafeMode activation

Traceability

- Section 13.4 Adversarial Input Requirements
- Section 11 Kernel Invariants
- Section 10 Hardening Framework
- L1/L2/L3 structural operators

13.5.1 Parameter Perturbation Tests

13.5.1 Parameter Perturbation Tests

Goal: Verify that small, bounded changes to UF kernel parameters do not cause instability, invariant violations, or discontinuities in UF processing.

Preconditions

- Python 3.9+ is installed.
- UF-Engine folder structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  config_variants/  
    inputs/  
    outputs/  
    logs/
```

- Baseline configuration:

```
config.yaml
```

- Parameter variants folder:

```
config_variants/
```

- Baseline input:

```
inputs/baseline.json
```

Required Data

Create parameter-perturbed configs with controlled deviations:

- `config_var_1.yaml` — change in τ_D
- `config_var_2.yaml` — change in $\psi_{\max}$
- `config_var_3.yaml` — perturb resonance weights λ_i
- `config_var_4.yaml` — perturb breathing thresholds $(B_{\min}, B_{\max})$
- `config_var_5.yaml` — perturb uncertainty weights $(\lambda_1, \lambda_2, \lambda_3)$

All perturbations MUST satisfy:

$$\|\Delta\Theta\| \leq \varepsilon_{\Theta}.$$

Setup

Store variant configs under:

```
config_variants/  
  config_var_1.yaml  
  config_var_2.yaml  
  config_var_3.yaml  
  config_var_4.yaml  
  config_var_5.yaml
```

Execution Steps

1. Run UF Engine with the baseline configuration:

```
python uf_engine.py --config config.yaml \  
  --input inputs/baseline.json \  
  --output outputs/baseline_param_result.json \  
  --log logs/baseline_param.log
```

2. For each variant config, run:

```
python uf_engine.py --config config_variants/config_var_1.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var1_result.json \  
  --log logs/test_13-3_var1.log
```

```
python uf_engine.py --config config_variants/config_var_2.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var2_result.json \  
  --log logs/test_13-3_var2.log
```

```
python uf_engine.py --config config_variants/config_var_3.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var3_result.json \  
  --log logs/test_13-3_var3.log
```

```
python uf_engine.py --config config_variants/config_var_4.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var4_result.json \  
  --log logs/test_13-3_var4.log
```

```
python uf_engine.py --config config_variants/config_var_5.yaml \  
  --input inputs/baseline.json \  
  --output outputs/test_13-3_var5_result.json \  
  --log logs/test_13-3_var5.log
```

3. From each output, record:

- "delta_S"
- "delta_U"
- "delta_Var_B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For all parameter variants:

- No UF Engine crashes or halts.
- "invariant_violations" must be:

[]
- DSF components remain bounded and finite.

- Breathing remains:

$$B_{\min} \leq B_k \leq B_{\max}$$

- Mosaic divergence satisfies:

$$C_k \leq C_{\max}$$

- Uncertainty satisfies:

$$U_k \leq U_{\max}$$

Pass / Fail Criteria

PASS if all variants satisfy:

- "invariant_violations" is empty.
- "delta_S" $\leq \kappa_{S\Theta}\varepsilon_{\Theta}$
- "delta_U" $\leq \kappa_{S\Theta}\varepsilon_{\Theta}$
- "delta_Var_B" remains within expected bounds.

FAIL if ANY variant shows:

- Non-empty invariant violations.
- Breathing instability.
- Mosaic divergence out of bounds.
- Uncertainty out of bounds.
- DSF blowup (NaN/INF).

Traceability

- Section 13.5 — Parameter Perturbation Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening Constraints
- L0–L4 operational equations

13.6.1 Memory Load and Schema Stress Tests

13.6.1 Memory Load and Schema Stress Tests

Goal: Verify that the UF engine maintains bounded schema growth, stable drift behavior, and invariant satisfaction when processing long structural sequences with sustained variation.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Long-run structural sequence generator exists:

```
scripts/generate_long_sequence.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Use the sequence generator to create a long structural field:

```
python scripts/generate_long_sequence.py \  
--length 5000 \  
--output inputs/test_13-4_longseq.json
```

The resulting dataset should include:

- 5,000+ timestamps,
- smooth and rough structural segments,
- multiple regime transitions,
- intermittent negative-space intervals.

Execution Steps

1. Run the UF engine on the long sequence:

```
python uf_engine.py --config config.yaml \  
--input inputs/test_13-4_longseq.json \  
--output outputs/test_13-4_longseq_result.json \  
--log logs/test_13-4_longseq.log
```

2. Inspect the output for:

- "schema_count"
- "max_C_k"
- "max_U_k"
- "max_B_k" and "min_B_k"
- "drift_measure"
- "invariant_violations"

Expected Results

UF MUST maintain stability under long-sequence load:

- Schema count bound:

$$S_{\text{count}} \leq S_{\text{max}}$$

- Schema entropy bound:

$$H_s \leq H_{\text{max}}$$

- Mosaic divergence:

$$C_k \leq C_{\text{max}}$$

- Drift bound:

$$\text{Drift}(T) \leq D_{\text{max}}$$

- Breathing dynamics:

$$B_{\text{min}} \leq B_k \leq B_{\text{max}}$$

- NO invariant violations.

Pass / Fail Criteria

PASS if:

- "invariant_violations" is []
- Schema count remains bounded
- Mosaic divergence stays within limits
- Drift remains stable (no spikes)
- Breathing does not collapse or blow up
- No NaN or INF in DSF or resonance metrics

FAIL if ANY of these occur:

- Schema count runaway
- Entropy runaway
- Breathing instability
- Drift spikes
- Divergence out of bounds
- Any invariant violation

Traceability

- Section 13.6 — Memory & Schema Stress Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening Constraints
- L1–L4 structural flow interactions

13.7.1 Multi-Lattice Conflict Tests

13.7.1 Multi-Lattice Conflict Tests

Goal: Verify that the UF engine maintains stable mosaic divergence, bounded uncertainty, and valid DSF behavior when lattice projections disagree (conflicting mosaic embeddings).

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Mosaic conflict generator exists:

```
scripts/generate_mosaic_conflict.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Generate mosaic-conflict inputs designed to induce disagreement between multiple lattices:

```
python scripts/generate_mosaic_conflict.py \  
--output inputs/test_13-5_conflict.json
```

Generated input must include:

- structural segments projected differently across lattices,
- mixed curvature patterns,
- alternating expansion/compression regions,
- at least one region where mosaic classes diverge.

Execution Steps

1. Run UF Engine:

```
python uf_engine.py --config config.yaml \
  --input inputs/test_13-5_conflict.json \
  --output outputs/test_13-5_conflict_result.json \
  --log logs/test_13-5_conflict.log
```

2. Inspect output fields:

- "max_C_k" — maximum mosaic divergence
- "max_U_k" — maximum uncertainty
- "max_B_k" and "min_B_k" — breathing envelope
- "invariant_violations"
- "dsf_summary"

Expected Results

UF MUST satisfy:

- Mosaic divergence bound:

$$C_k \leq C_{\max}$$

- Uncertainty bound:

$$U_k \leq U_{\max}$$

- Breathing stability:

$$B_{\min} \leq B_k \leq B_{\max}$$

- NO invariant violations:

[]

- DSF signals remain finite (no NaN/INF).

Pass / Fail Criteria

PASS if:

- Mosaic divergence remains bounded,
- Uncertainty remains within limits,
- Breathing stays within allowed envelope,
- No invariant violations occur,
- DSF components remain finite.

FAIL if ANY of these occur:

- $C_k > C_{\max}$,
- $U_k > U_{\max}$,
- breathing instability or collapse,
- any invariant violation,
- DSF instability or divergence (NaN/INF).

Traceability

- Section 13.7 — Multi-Lattice Conflict Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening Framework
- L1 Mosaic projection operators
- L2 Uncertainty metrics

13.8.1 Breathing–Self-Organization Synchronization

13.8.1 Breathing–Self-Organization Synchronization

Goal: Verify that breathing dynamics (bandwidth modulation) remain synchronized with schema evolution under different structural regimes, without causing runaway growth, collapse, or instability.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Regime scenario generator exists:

```
scripts/generate_regime_scenarios.py
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Two structural regimes MUST be generated:

- Regime A — high activity (greater drift, faster transitions)
- Regime B — low activity (stable and slower transitions)

Generation commands:

```
python scripts/generate_regime_scenarios.py \  
  --mode A --output inputs/test_13-6_regimeA.json  
  
python scripts/generate_regime_scenarios.py \  
  --mode B --output inputs/test_13-6_regimeB.json
```

Execution Steps

1. Run UF on Regime A:

```
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-6_regimeA.json \  
  --output outputs/test_13-6_regimeA_result.json \  
  --log logs/test_13-6_regimeA.log
```

2. Run UF on Regime B:

```
python uf_engine.py --config config.yaml \  
  --input inputs/test_13-6_regimeB.json \  
  --output outputs/test_13-6_regimeB_result.json \  
  --log logs/test_13-6_regimeB.log
```

3. For each result, extract:

- "breathing_amplitude_series" or summary:
 - "max_A_B"
 - "mean_A_B"
- "schema_mutation_rate"
- "drift_measure"
- "invariant_violations"

Expected Results

Under Regime A:

- Breathing amplitude SHOULD exceed Regime B.
- Schema mutation rate SHOULD exceed Regime B.

Under Regime B:

- Breathing amplitude SHOULD be lower than Regime A.
- Schema mutation rate SHOULD be lower than Regime A.

For BOTH regimes:

- NO invariant violations.
- Breathing amplitude MUST remain in bounds:

$$A_B(k) \leq \alpha_1 D_T(k) + \alpha_2 |\Delta R(k)| + B_0$$

- Schema mutation MUST remain bounded:

$$\mu_s(k) \leq \mu_{s,\max}$$

- Combined stability condition holds:

$$A_B(k) \mu_s(k) \leq \beta_{\max}$$

Pass / Fail Criteria

PASS if ALL of the following hold:

- Relative differences between Regime A and B match expectations.
- No invariant violations occur.
- Breathing remains stable (no blowup or collapse).
- Schema mutation rate is bounded.

FAIL if ANY of the following occur:

- Breathing runaway.
- Schema mutation instability.
- Invariant violations.
- Regime A and B do not exhibit the expected relative behavior.

Traceability

- Section 13.8 — Breathing–Self-Organization Synchronization
- Section 11 — Kernel Invariants
- Section 10 — Hardening Framework
- L2, L3 structural transitions

13.9.1 SES Evolution Tests

13.9.1 SES Evolution Tests

Goal: Verify that SES (Structural Encryption System) remains structurally compatible and invertible under bounded changes to SES-related parameters and across version transitions, while maintaining invariant and hardening compliance.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  ses_config/  
    inputs/  
    outputs/  
    logs/  
    scripts/
```

- SES-compatible UF Engine implementation exists (encrypt/decrypt paths active).
- SES configuration versions exist:

```
ses_config/ses_v1.yaml  
ses_config/ses_v2.yaml
```

- Baseline structural input exists:

```
inputs/baseline.json
```

Required Data

Two SES versions MUST be exercised:

- `ses_v1.yaml` — baseline SES parameters.
- `ses_v2.yaml` — perturbed SES parameters:
 - bounded mask changes,
 - bounded permutation index changes,
 - bounded shift-vector changes.

All deviations MUST satisfy:

$$\|\Delta\Theta_{\text{SES}}\| \leq \varepsilon_{\Theta}.$$

Execution Steps

1. Encrypt and decrypt using SES v1:

```
python uf_engine.py --config config.yaml \
--ses-config ses_config/ses_v1.yaml \
--input inputs/baseline.json \
--output outputs/test_13-7_ses_v1_result.json \
--log logs/test_13-7_ses_v1.log
```

2. Encrypt and decrypt using SES v2:

```
python uf_engine.py --config config.yaml \
--ses-config ses_config/ses_v2.yaml \
--input inputs/baseline.json \
--output outputs/test_13-7_ses_v2_result.json \
--log logs/test_13-7_ses_v2.log
```

3. For each SES version result, extract:

- "structural_equivalence" (boolean)
- "delta_K" (key drift measure)
- "invariant_violations"
- "ses_status"

Expected Results

For both SES versions:

- Structural equivalence is preserved:

$$\text{StructEq}(\Gamma_k, \Gamma'_k) = 1$$

- Key drift remains bounded:

$$\Delta K_k \leq \kappa_K \varepsilon_\Theta$$

- NO invariant violations:

[]

- SES status MUST be reported as "ok".

Pass / Fail Criteria

PASS if all of the following hold:

- "structural_equivalence" is true.
- $\Delta K_k \leq \kappa_K \varepsilon_\Theta$.
- No invariant violations occur.
- SES status is "ok" for both versions.

FAIL if ANY occur:

- Structural equivalence fails,
- Key drift exceeds bound,
- Any invariant violation,
- SES status is not "ok".

Traceability

- Section 13.9 — SES Evolution Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening & Compatibility Constraints
- SES-Core (Section 15) — encryption and redaction requirements

13.10.1 Cross-Domain Transfer Tests

13.10.1 Cross-Domain Transfer Tests

Goal: Verify that the UF engine maintains stable behavior, bounded invariants, and valid DSF outputs when identical structural patterns are embedded across domains of different dimensionality and scaling.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Cross-domain generator script exists:

`scripts/generate_cross_domain.py`

- `config.yaml` from UF-Spec v1.4.0 baseline is present.

Required Data

Generate structurally similar inputs for multiple domains:

- Domain 1: $d_1 = 1$
- Domain 2: $d_2 = 3$
- Domain 3: $d_3 = 5$

```
python scripts/generate_cross_domain.py --dim 1 --output inputs/test_13-8_dim1.json
python scripts/generate_cross_domain.py --dim 3 --output inputs/test_13-8_dim3.json
python scripts/generate_cross_domain.py --dim 5 --output inputs/test_13-8_dim5.json
```

Execution Steps

1. Run UF Engine on each domain:

```
python uf_engine.py --config config.yaml \
  --input inputs/test_13-8_dim1.json \
  --output outputs/test_13-8_dim1_result.json \
  --log logs/test_13-8_dim1.log

python uf_engine.py --config config.yaml \
  --input inputs/test_13-8_dim3.json \
  --output outputs/test_13-8_dim3_result.json \
  --log logs/test_13-8_dim3.log

python uf_engine.py --config config.yaml \
  --input inputs/test_13-8_dim5.json \
  --output outputs/test_13-8_dim5_result.json \
  --log logs/test_13-8_dim5.log
```

2. Extract from each result:

- "delta_S"
- "delta_U"
- "delta_Var.B"
- "invariant_violations"
- "dsf_summary"

Expected Results

For all domains:

- Stability deviation satisfies:

$$\Delta S_X \leq \kappa_{SD} \cdot d_X$$

- Mosaic divergence:

$$C_k \leq C_{\max}$$

- Schema entropy:

$$H_s \leq H_{\max}$$

- Drift bounds:

$$\text{Drift}(T_X) \leq D_{\max, X}$$

- Breathing variance:

$$\text{Var}(B_X) \leq \text{Var}(B_0) + \kappa_{BD} \cdot d_X$$

- DSF values remain finite and bounded.
- Invariant violations MUST be:

□

Pass / Fail Criteria

PASS if all domains satisfy:

- $\Delta S_X \leq \kappa_{SD} d_X$
- $C_k \leq C_{\max}$
- $H_s \leq H_{\max}$
- $\text{Drift}(T_X) \leq D_{\max, X}$
- Breathing variance stable
- No invariant violations
- DSF contains no NaN or INF values

FAIL if ANY domain exhibits:

- excessive ΔS_X
- mosaic divergence out of bounds
- schema entropy runaway
- drift runaway
- breathing instability
- invariant violation

Traceability

- Section 13.10 — Cross-Domain Transfer Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening Model
- L0–L4 operator consistency

13.11.1 Global Stability Shock Tests

13.11.1 Global Stability Shock Tests

Goal: Verify that the UF engine remains stable, properly triggers SafeMode when required, respects invariants, and returns to baseline stability after exposure to extreme structural shocks, including: lattice breaks, resonance spikes, catastrophic variance bursts, parameter jumps, and structural regime breaks.

Preconditions

- Python 3.9+ is installed.
- UF-Engine directory structure exists:

```
UF-Engine/  
  uf_engine.py  
  config.yaml  
  inputs/  
  outputs/  
  logs/  
  scripts/
```

- Shock scenario generator exists:

```
scripts/generate_shock_scenarios.py
```

- Baseline input exists:

```
inputs/baseline.json
```

- config.yaml from UF-Spec v1.4.0 is present.

Required Data

Generate five distinct shock scenarios:

- Shock A — Lattice break
- Shock B — Resonance spike
- Shock C — Catastrophic variance burst

- Shock D — Extreme parameter drift
- Shock E — Structural regime break

```
python scripts/generate_shock_scenarios.py --type A --output inputs/test_13-9_shock_A.json
python scripts/generate_shock_scenarios.py --type B --output inputs/test_13-9_shock_B.json
python scripts/generate_shock_scenarios.py --type C --output inputs/test_13-9_shock_C.json
python scripts/generate_shock_scenarios.py --type D --output inputs/test_13-9_shock_D.json
python scripts/generate_shock_scenarios.py --type E --output inputs/test_13-9_shock_E.json
```

Execution Steps

1. Run UF Engine on each shock scenario:

```
python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_A.json \
--output outputs/test_13-9_shock_A_result.json \
--log logs/test_13-9_shock_A.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_B.json \
--output outputs/test_13-9_shock_B_result.json \
--log logs/test_13-9_shock_B.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_C.json \
--output outputs/test_13-9_shock_C_result.json \
--log logs/test_13-9_shock_C.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_D.json \
--output outputs/test_13-9_shock_D_result.json \
--log logs/test_13-9_shock_D.log

python uf_engine.py --config config.yaml \
--input inputs/test_13-9_shock_E.json \
--output outputs/test_13-9_shock_E_result.json \
--log logs/test_13-9_shock_E.log
```

2. For each output file, extract:

- "post_shock_S"
- "safe_mode"
- "invariant_violations"
- "drift_measure"
- "max_C_k", "max_U_k", "max_B_k"
- "recovery_S"

Expected Results

For all shock types:

- Stability must satisfy:

$$S(\text{UF}_{\text{shock}}) \geq 0$$

- SafeMode MUST activate when:

$$S(\text{UF}_{\text{shock}}) < S_{\text{min}}$$

- Invariants remain bounded:

$$C_k \leq C_{\text{max}}, \quad U_k \leq U_{\text{max}}, \quad B_{\text{min}} \leq B_k \leq B_{\text{max}}$$

- After a recovery window:

$$\text{recovery_S} \rightarrow S(\text{UF}_0)$$

i.e., return to baseline stability.

Pass / Fail Criteria

PASS if ALL of the following hold:

- post-shock stability is non-negative,
- SafeMode activates exactly when required,
- invariants remain satisfied,
- drift remains bounded,
- DSF remains finite,
- recovery converges toward baseline.

FAIL if ANY occur:

- missing or incorrect SafeMode activation,
- invariants violated,
- drift runaway,
- breathing runaway,
- DSF blowup (NaN/INF),
- failure to recover stability.

Traceability

- Section 13.11 — Global Stability Shock Requirements
- Section 11 — Kernel Invariants
- Section 10 — Hardening Triggers (HTC)
- Structural flow through L0–L4

Validation Matrix

Test ID	Description	Pass Criteria Summary
13.0.0	Global Protocol Standards	Deterministic protocol structure satisfied
13.3.1	Noise and Perturbation Robustness	$\Delta S(\varepsilon) \leq \kappa_S \varepsilon$, $\Delta U(\varepsilon) \leq \kappa_U \varepsilon$, $\Delta \text{Var}(B_\varepsilon) \leq \kappa_B \varepsilon$
13.4.1	Adversarial Structured Inputs	All structural bounds (stability, uncertainty, mos
13.5.1	Parameter Perturbation	$\Delta S_\Theta \leq \kappa_{S\Theta} \varepsilon_\Theta$
13.6.1	Memory and Schema Stress	$S_{\text{count}} \leq S_{\text{max}}$, $H_s \leq H_{\text{max}}$, $C_k \leq C_{\text{max}}$, $\text{Drift}(T)$
13.7.1	Multi-Lattice Mosaic Conflict	$C_k \leq C_{\text{max}}$, $U_k \leq U_{\text{max}}$, $S_{\text{count}} \leq S_{\text{max}}$, $S(\text{UF}, k)$
13.8.1	Breathing–Self-Organization Synchronization	$A_B(k) \mu_s(k) \leq \beta_{\text{max}}$ and $S(\text{UF}, k) \geq S_{\text{min}}$ for all
13.9.1	SES Evolution	$\text{StructEq}(\Gamma_k, \Gamma'_k) = 1$, $\Delta K_k \leq \kappa_K \varepsilon_\Theta$, $S(\text{UF}_{\Theta'}) \geq$
13.10.1	Cross-Domain Transfer	$\Delta S_X \leq \kappa_{SD} d_X$, $C_k \leq C_{\text{max}}$, $H_s \leq H_{\text{max}}$, $\text{Drift}(T)$
13.11.1	Global Stability Shock	$S(\text{UF}_{\text{shock}}) \geq 0$, SafeMode activates iff $S(\text{UF}_{\text{shock}})$

Part IV — Adapters

14 Adapter Architecture Overview

This section provides a structural overview of how domain adapters integrate with the Unified Framework (UF) Kernel. Adapters define how external domains map their input data into UF’s Layer 0 input series and how UF outputs are interpreted safely within each domain.

The Adapter Layer SHALL:

- specify valid input formats for the domain,
- define UF invocation constraints for the domain,
- define redaction and SES–Core usage rules,
- specify which UF Summary fields MAY be consumed downstream,
- define requirements for domain-specific validation,
- define governance-layer interpretation rules,
- enforce domain binding and version binding,
- prohibit any reconstruction of UF internal operators.

The following adapters are defined in UF–Spec v1.4.0:

- **15** — SES–Core (UF Encryption Platform),
- **16** — Financial Domain Adapter,
- **17** — Future Adapters (reserved).

Adapters MUST NOT modify UF Kernel logic or internal computation. They operate strictly as boundary controllers between the Kernel and external systems.

15 SES–Core (UF Encryption Platform)

C.1 Purpose and Scope

Normative Status: SES–Core v2.0 is the mandatory encryption and redaction layer for the Unified Framework. All UF Structural Payloads (UF–SP) SHALL be encrypted before leaving the kernel boundary.

SES–Core defines:

- the UF Structural Payload,
- deterministic canonicalization rules,
- the SES Envelope format,
- structural redaction policy,
- domain binding rules,
- two–layer key architecture,
- AEAD–SIV authenticated encryption,
- deterministic structural DRBG,
- security and compliance requirements.

C.2 Terminology and Definitions

- **UF–SP:** The complete internal structural output of the UF kernel.
- **SES Envelope:** Header, summary and ciphertext.
- **Domain Parameters:** Symbol, window, timeframe, UF version, SES version, tenant identifier.
- **Structural Summary:** Whitelisted metrics allowed in plaintext.
- **Root Key:** Stored in HSM, used only for key derivation.
- **Domain Key:** Derived symmetric key for AEAD–SIV.
- **Structural DRBG:** Deterministic pseudorandom generator.

C.3 Threat Model

Adversarial Goals

- Recover internal UF structure,
- Infer kernel operators or dynamics,
- Tamper with UF outputs,
- Manipulate domain parameters,
- Replay envelopes across domains.

Out-of-Scope Threats

- Operating system compromise,
- HSM compromise,
- Hardware tampering,
- Side-channel attacks.

C.4 UF Structural Payload

UF-SP SHALL include:

- SEV fields,
- L1 gate boundaries,
- L2 interpretive vectors,
- L3 resonance fields,
- L4 DSF vectors,
- Section 13 validation metrics,
- Hardening diagnostics.

C.5 Redaction Policy

Plaintext output SHALL be limited to:

- Stability score,
- Robustness score,
- Gate stability score,

- Directional stability score,
- DSF stability,
- Collapse indicators,
- SafeMode indicator.

Everything else MUST be encrypted.

C.6 SES Envelope Format

Structure

```
SESEnvelope {
    header: { ... },
    summary: { ... },
    ciphertext: bytes
}
```

C.7 Two-Layer Key Architecture

Root Key

Stored in HSM; used exclusively for HKDF.

Domain Key

$$K_{\text{domain}} = \text{HKDF}(K_{\text{root}}, \text{DomainParams})$$

C.8 Structural DRBG

$$\text{DRBG}(K_{\text{domain}}, UF_SP) \rightarrow R$$

C.9 AEAD-SIV Encryption

$$C = \text{SIV_Encrypt}(K_{\text{domain}}, P, A)$$

C.10 Algorithms

Encode

```
function SES_Encode(UF_SP, DomainParams):
    header = DomainParams
    summary = Redact(UF_SP)
```

```

K = HKDF(K_root, DomainParams)
A = Serialize(header, summary)
P = Serialize(UF_SP)
C = SIV_Encrypt(K, P, A)
return Envelope(header, summary, C)

```

Decode

```

function SES_Decode(Envelope, DomainParams):
    K = HKDF(K_root, DomainParams)
    A = Serialize(Envelope.header, Envelope.summary)
    P = SIV_Decrypt(K, Envelope.ciphertext, A)
    return Deserialize(P)

```

C.11 Domain Binding Rules

Valid decryption requires:

$$\text{DomainParams}' = \text{DomainParams}$$

C.12 Security Properties

- Confidentiality,
- Integrity,
- Inference resistance,
- Replay resistance,
- Version isolation,
- Tenant isolation.

C.13 Implementation Requirements

Implementations MUST:

- use canonical JSON for UF-SP,
- use HKDF for key derivation,
- use AEAD-SIV for encryption,
- use deterministic DRBG output,
- enforce strict domain binding,
- log SES and UF versions,
- reject malformed envelopes.

16 Financial Domain Adapter

D.0 Purpose and Scope

The Financial Domain Adapter defines how the UF Kernel SHALL be used with financial market data. It specifies:

- which financial inputs are valid for UF,
- how those inputs are mapped into UF input series,
- how UF evaluations are configured in this domain,
- which UF outputs MAY be exposed to the Tao Financial Engine (TFE),
- how SES-Core encryption MUST be applied to protect UF-SP,
- how governance logic MUST interpret UF summaries.

UF is a structural analysis engine, not a price predictor. The Financial Adapter enforces that boundary.

D.1 Financial Input Domain Specification

The Financial Adapter accepts only daily OHLCV data:

$$\{(ts_i, O_i, H_i, L_i, C_i, V_i) \mid i = 1 \dots n\}$$

with the following requirements:

- Timestamps strictly increasing.
- $H_i \geq L_i$ and $O_i, C_i \in [L_i, H_i]$.
- $V_i \geq 0$.
- Bars sorted chronologically.
- If adjusted data is used, all fields MUST be consistently adjusted.

Missing OHLC fields SHALL cause input rejection. Daily resolution is normative in this version.

D.2 UF Invocation Rules in the Financial Domain

UF SHALL receive as input the closing-price series:

$$\mathcal{I} = \{(t_i, C_i) \mid i = 1 \dots n\}$$

Only C_i is passed to L0. UF SHALL NOT directly consume O, H, L, V .
Evaluation windows:

- MUST have minimum length (e.g. 90 bars).
- MUST be contiguous in trading days.
- MUST be treated as independent (no streaming).

UF configuration in finance:

- standard SEV tuple,
- fixed global gate threshold,
- noise levels as in D.2.4,
- full hardening (Section 23) enabled.

D.2.4 Noise Sensitivity Parameters

UF financial evaluations SHALL use:

$$\sigma_{pct} \in \{0.1\%, 0.2\%, 0.5\%, 1.0\%, 2.0\%\}.$$

These perturbations are used to compute:

- Gate Stability Score (GSS),
- Directional Stability Score (DSS),
- DSF Stability Score,
- composite metrics $S(UF)$ and $R(UF)$.

D.3 TFE-Side Summary Requirements

Only the following SES Summary fields MAY be visible to TFE:

- $S(UF)$,
- $R(UF)$,
- GSS, DSS, DSF Stability,
- collapse flags (DSF collapse, Composite_S_low, Composite_R_low),

- SafeMode indicator.

No SEV, gate boundaries, resonance curves, DSF series or other internal UF values MAY be exposed.

TFE SHALL interpret these metrics as:

- $S(UF)$: structural stability (not direction),
- $R(UF)$: robustness (not probability of profit),
- stability metrics: coherence of structural behavior,
- collapse flags / SafeMode: structural anomaly indicators.

UF outputs SHALL NOT be treated as predictions.

D.4 Multi-Asset and Multi-Class Support

Supported classes in this version:

- equities,
- ETFs,
- broad indices,
- crypto assets with OHLCV-like aggregates.

UF MUST treat all asset classes identically:

- same SEV construction,
- same gate logic,
- same interpretive and resonance rules,
- same DSF computation,
- same validation and hardening logic.

No asset-specific branches MAY exist inside UF.

Each asset SHALL be evaluated independently; UF MUST NOT share internal states across assets.

D.5 Domain Binding Requirements in the Financial Adapter

SES-Core domain parameters in finance:

(symbol, start_date, end_date, timeframe, uf_version, ses_version, org_id).

SES Envelopes MUST decrypt only when all parameters match exactly. Any mismatch SHALL cause decryption failure.

Symbol Binding prevents cross-asset misuse. Window Binding prevents replay across different time ranges. Timeframe Binding prevents daily vs. intraday confusion. Version Binding isolates UF and SES versions. Tenant Binding isolates multi-tenant TFE deployments.

D.6 SES-Core Usage Requirements in the Financial Domain

UF-SP MUST:

- always be encrypted via SES-Core,
- never be stored or transmitted in plaintext,
- never be logged or cached,
- never be passed directly into TFE.

Only SES Summary MAY enter TFE logic.
Persistent storage of UF results MUST use:

- SES Envelopes (ciphertext),
- encryption-at-rest at the infrastructure level,
- retained domain parameters and versions for audit.

D.7 Governance Layer (TFE) Interpretation Logic

TFE MUST:

- treat UF as a structural analyzer only,
- use $S(UF)$, $R(UF)$ and stability scores for qualitative flags (e.g. stable/fragile),
- use collapse flags and SafeMode as hard blockers (no advice),
- NOT derive directional predictions from UF outputs,

- NOT compute probability of gain/loss from UF metrics.

Thresholds (e.g. “stable if $S(UF) \geq S_{\text{high}}$ ”) MAY be defined, but MUST be based only on SES Summary and MUST NOT attempt to reconstruct UF internals.

D.8 Performance and Deployment in Market Systems

UF + SES-Core + TFE MUST:

- support batch evaluations over many assets,
- treat each UF evaluation as independent,
- allow horizontal scaling (multiple compute nodes),
- ensure deterministic outputs independent of deployment topology.

Caching MAY store SES Envelopes or SES Summaries keyed by domain parameters, but MUST NOT cache UF-SP or partial UF states.

D.9 Validation Requirements for the Financial Adapter

Section 13 validation in the financial domain MUST:

- use the σ_{pct} values defined in D.2.4,
- reject windows shorter than the minimum length,
- ensure stability metrics in SES Summary match validation outputs,
- confirm deterministic reproducibility across runs.

UF outputs MUST NOT be considered valid for TFE if:

- collapse flags are active,
- validation metrics fall below configured thresholds,
- domain parameters or versions do not match expectations.

D.10 Appendix D Completeness and Specification Closure

Appendix 16 defines the complete and exclusive rules for using UF in the financial domain. Any deployment of UF in financial systems **MUST**:

- conform to input rules (D.1),
- follow invocation rules (D.2),
- restrict outputs to SES Summary (D.3),
- respect multi-asset rules (D.4),
- enforce domain-binding (D.5),
- apply SES–Core exactly as specified (D.6),
- follow governance interpretation rules (D.7),
- satisfy performance constraints (D.8),
- satisfy validation rules (D.9).

Non-conformant usage **SHALL** be considered outside the scope of UF–Spec v1.4.0.

17 Future Adapters

Future domain adapters (e.g., medical, robotics, cognitive systems, or additional financial verticals) **SHALL** be specified in versioned extensions to UF–Spec. This section currently serves as a placeholder to reserve numbering and TOC structure for future Part IV additions.

No normative content is defined in Section 17 for UF–Spec v1.4.0.

Part V — Governance

18 Versioning Rules

This section defines the formal versioning policy for the Unified Framework (UF), SES–Core, and all domain adapters. Versioning rules ensure that all components interact safely, reproducibly, and without ambiguity.

18.1 Purpose of Versioning

Versioning defines:

- which UF components may change,
- how changes must be documented,
- which changes require a major, minor, or patch version increment,
- how adapters and SES-Core declare compatibility.

Versioning enforces determinism and prevents silent semantic drift.

18.2 Semantic Versioning Model

UF uses three-part semantic versioning:

$vX.Y.Z$

- **Major (X)** — Any change to Kernel Operators L0–L4, stability metrics, invariants, resonance gating, decision dynamics, or SES-Core canonicalization.
- **Minor (Y)** — Additive, backward-compatible extensions such as new adapters, new validation tests, or clarifying specification text.
- **Patch (Z)** — Error corrections, documentation cleanup, or implementation clarifications that do not alter semantics.

18.3 Kernel Version Isolation

A UF instance MUST NOT:

- mix components from different Kernel versions,
- bind SES-Core versions lower than required by the Kernel,
- bind domain adapters not explicitly marked as compatible.

Version isolation ensures that all validations, hardening rules, and invariants apply correctly to the running system.

18.4 SES-Core Version Compatibility

SES-Core versions MUST be explicitly declared compatible with:

- the UF Kernel version,

- the domain adapter version,
- the externalization schema.

If SES-Core introduces new redaction fields or removes old fields, a MINOR version increment is required.

If SES-Core modifies canonicalization or StructEq, a MAJOR version increment is required.

18.5 Adapter Version Compatibility

Domain adapters MUST:

- declare the range of UF versions they support,
- declare the SES-Core versions they support,
- reject Kernel outputs from incompatible versions.

Adapters MUST NOT reinterpret UF internal representations (see Governance, Section 22).

18.6 Required Version Increments

A MAJOR version increment is required for:

- changes to Kernel operators L0–L4,
- changes to Hardening triggers or actions,
- changes to invariants or stability metrics,
- changes to StructEq or SES-Core canonicalization,
- changes to L2 regime definitions,
- changes to L3 resonance functional.

A MINOR increment is required for:

- new adapters,
- new validation tests,
- new SES-Core summary fields.

A PATCH increment is required for:

- documentation corrections,
- mathematical clarifications,
- implementation detail expansions.

18.7 Summary

Versioning ensures:

- deterministic UF behavior,
- compatibility across Kernel, SES-Core, and adapters,
- reproducibility and auditability,
- correct interpretation of all validation results.

A UF system is compliant only when all version constraints are satisfied.

19 Compliance & Reproducibility

Compliance and reproducibility define the operational and regulatory requirements under which implementations of the Unified Framework (UF) MUST operate. These requirements ensure that UF remains deterministic, auditable, version-stable, and structurally correct across all domains and deployments. Compliance rules bind every component of the system, including the Kernel, SES-Core, domain adapters, and validation suites.

This section defines the normative obligations for conformant UF systems.

19.1 Deterministic Behavior Requirement

All conformant UF implementations MUST be deterministic:

- For identical inputs $\mathcal{I}$ and configuration parameters, two evaluations MUST produce bit-identical UF outputs (within floating-point tolerances defined in Section 11).
- No component of UF MAY depend on hidden state or history.
- No adaptive behavior, learning, parameter updates, or contextual modification MAY occur during evaluation.

Determinism is the foundation of reproducibility and SES-Core domain-binding.

19.2 Version Isolation Requirement

Implementations MUST respect strict version isolation:

- UF version identifiers MUST change whenever Kernel operators or internal representations change.

- SES–Core version identifiers **MUST** change whenever the UF Structural Payload (UF–SP) format or canonicalization rules change.
- Domain adapters **MUST** declare the UF and SES–Core versions they are compatible with.

Silent behavioral drift across versions is prohibited.

19.3 Validation Conformance Requirement

A UF implementation **MUST** pass all validation procedures defined in Section 13 before it can be considered compliant. Specifically:

- All stability metrics **MUST** match the canonical definitions.
- Structural invariance **MUST** hold under perturbation.
- Collapse modes **MUST** be detected consistently.
- SafeMode **MUST** activate under the conditions defined in Section 23.

Failure on any validation test indicates non-compliance.

19.4 SES–Core Compliance Requirement

All UF outputs external to the Kernel boundary **MUST** be protected by SES–Core (Section 15). Implementations **MUST** satisfy:

- canonical serialization of UF–SP,
- correct construction of SES Envelopes,
- correct application of domain-binding rules,
- rejection of malformed headers and mismatched domains,
- correct AEAD–SIV encryption and authentication,
- correct use of version identifiers.

Any exposure of unencrypted UF–SP constitutes non-compliance.

19.5 Domain Adapter Compliance Requirement

Domain adapters (Part IV) MUST comply with the following:

- Domain inputs MUST satisfy the domain’s structural definition.
- UF invocation MUST follow domain-specific constraints.
- Only SES Summary fields MAY be externalized.
- Domain adapters MUST NOT reinterpret UF internal operators.
- Adapter behavior MUST remain version-stable.

Adapters define the legal usage boundary of UF for each domain.

19.6 Reproducibility Requirements

UF evaluations MUST satisfy two forms of reproducibility:

- **Run-to-run reproducibility:** repeating the same evaluation MUST yield identical results.
- **Implementation reproducibility:** different implementations MUST produce equivalent outputs for identical inputs.

Additionally:

- Randomization is prohibited inside the Kernel.
- All noise used for validation MUST be externally generated.
- The Structural DRBG MAY only be used for SES–Core encryption.

19.7 Auditability Requirements

A conformant implementation MUST:

- log UF version, SES–Core version, and adapter version,
- log SES Envelope headers (but not ciphertext),
- retain domain parameters used in evaluations,
- retain validation results,
- guarantee that logs do not contain UF–SP.

Auditability ensures long-term traceability of UF evaluations.

19.8 Compliance Summary

A UF implementation is compliant with UF–Spec v1.4.0 if and only if:

- it implements the Kernel exactly as specified,
- it satisfies all invariants (Section 8),
- it satisfies all stability properties (Section 9),
- it passes all validation tests (Section 13),
- it applies SES–Core exactly as defined (Section 15),
- it uses domain adapters correctly (Part IV),
- it follows governance rules (Section 22),
- it satisfies all reproducibility and auditability requirements.

Non-conformant implementations MUST NOT be deployed or used for interpretation in any domain.

20 Glossary

Unified Framework (UF): Deterministic structural computation framework defined by Layers L0–L4.

Normalized Structural Field (NSF): The sequence of Structural Evaluation Vectors (SEVs) produced by L0.

Structural Evaluation Vector (SEV):

$$SEV(t) = (F(t), \Delta F(t), \sigma(t), \kappa(t), r(t), N(t)).$$

Gate (G_k): Contiguous structural interval defined by L1’s deviation thresholding.

TVR Vector: Time–Volume–Relevance descriptor (T_k, V_k, R_k) for each gate.

Mosaic Projection (P_ℓ): Structural quantization mapping each gate into discrete mosaic coordinates.

Mosaic Divergence (C_k): Number of distinct mosaic projections across all lattices.

Interpretive Structural Field (ISF_k): L2 interpretive representation:

$$(w_k, CV_k, S_k, Reg_k, U_k, IAS_k).$$

Resonance ($R(k)$): Scalar structural coherence measure in L3.

Unified Resonance Field (URF_k):

$$URF_k = g_k \cdot R(k)$$

Decision Surface Vector (DSF_k): L4 decision-state vector:

$$(D_k, M_k, R_k, U_k^*, C_k, P_k, B_k).$$

SES-Core: The encryption and redaction layer required for all UF externalization.

UF-SP (UF Structural Payload): UF-internal representation that **MUST** NOT be exposed directly.

SafeMode: Kernel safety mode triggered by the Hardening Model (Section 10).

Hardening: Set of Trigger Conditions (HTCs), Actions (HAs), and Recovery Rules (HRRs) that enforce bounded behavior in L0-L4.

21 References

This section lists the normative and informative references relevant to the Unified Framework (UF) and its associated components, including the Kernel, SES-Core, domain adapters, validation methodology, and cryptographic primitives. Where applicable, formal citations correspond to industry standards or peer-reviewed research.

21.1 Normative References

These references define required standards or specifications that UF implementations **MUST** comply with.

- **UF-Spec v1.4.0:** All requirements, invariants, stability metrics, and structural operators defined in this specification.
- **Section 11: Formal Mathematics:** Canonical mathematical definitions of UF operators, invariants, and metrics.
- **AES-SIV (RFC 5297):** Definition of deterministic authenticated encryption used in SES-Core.
- **HKDF (RFC 5869):** Key derivation function used for UF domain keys in SES-Core.
- **NIST SP 800-90A:** Deterministic random bit generators for cryptographic applications (pertains to Structural DRBG requirements).
- **NIST SP 800-38D:** GCM mode (informational reference for authenticated encryption; not used directly but relevant for cryptographic context).

21.2 Informative References

These references provide background concepts but are not required for conformance.

- Dwork & Roth (2014): *The Algorithmic Foundations of Differential Privacy*. Relevant conceptually for understanding predictable perturbation sensitivity (not used directly).
- Boyd & Vandenberghe (2004): *Convex Optimization*. Provides mathematical background for structural invariants and stability concepts.
- Tao (2006): *Nonlinear Dispersive Equations*. Background for resonance and structural wave interpretation (UF does not implement PDEs but leverages structural analogies conceptually).

21.3 UF Internal Cross-References

The following internal references define authoritative content within the UF-Spec:

- **Section 1:** Introduction to UF and its conceptual boundaries.
- **Section 2:** Mathematical Overview (non-normative summary).
- **Section 8:** Invariants.
- **Section 9:** Stability Metrics.
- **Section 12:** Academic Validation Layer.
- **Section 13:** Operational Validation Suite.
- **Section 15:** SES-Core Encryption Platform Specification.
- **Section 16:** Financial Domain Adapter Specification.
- **Section 22:** Governance and Versioning Rules.
- **Glossary:** Terminology and definitions.

21.4 Reference Format

This specification does not mandate a particular bibliography format. Normative references SHOULD be stable, versioned, and publicly accessible. UF implementations MAY provide additional references in implementation documentation, but such references are non-normative unless incorporated into a future revision of UF-Spec.

21.5 Completeness of References

This list is expected to expand as future UF adapters, cryptographic modules, and domain applications are introduced. All additions **MUST** be approved at the specification level and versioned accordingly.

22 Governance and Versioning Rules

This section defines the governance model for the Unified Framework (UF), including versioning rules, change control, implementation constraints, and obligations for adapters and external systems. Governance ensures that UF remains deterministic, reproducible, audit-ready, and domain-safe across all deployments and implementations.

22.1 Purpose of Governance

The UF Kernel, SES-Core, and domain adapters **MUST** operate under strict governance constraints to:

- preserve structural correctness,
- ensure deterministic behavior,
- maintain version isolation,
- prevent unauthorized reinterpretation of UF internals,
- ensure valid use of SES-Core redaction and encryption,
- guarantee compliance and auditability.

Governance defines what implementations **MAY** do, **MUST** do, and **MUST NOT** do.

22.2 Versioning Policy

UF version numbers **MUST** increment whenever:

- Kernel operators (L0–L4) are modified,
- internal structural representations change,
- stability metrics or invariants change,
- SES-Core canonicalization rules change,
- collapse detection rules change,
- validation requirements change.

Minor or patch version updates MAY occur for:

- text clarifications,
- non-semantic specification corrections,
- documentation improvements.

Version drift without version number changes is prohibited.

22.3 Canonical Implementation Requirements

A conformant implementation MUST:

- implement L0–L4 EXACTLY as defined in UF–Spec v1.4.0,
- obey all invariants in Section 8,
- compute stability metrics exactly as defined in Section 9,
- pass all validation tests in Section 13,
- apply the Hardening Model (Section 10) without deviation,
- externalize UF–SP ONLY through SES–Core (Section 15),
- use domain adapters for all interpretation (Part IV),
- maintain deterministic, run-to-run equivalence,
- reject malformed inputs, malformed SES Envelopes, or invalid domain bindings.

No modifications or optimizations MAY alter the logical behavior of Kernel operators.

22.4 Change Control

Any change to the UF Kernel, SES–Core, or domain adapter specifications MUST:

- be documented at the specification level,
- be accompanied by a new version identifier,
- include updated validation procedures,
- include updated governance rules if applicable.

Implementations MUST NOT introduce unapproved or undocumented changes.

22.5 Forbidden Behaviors

The following behaviors are explicitly prohibited:

- leaking UF-SP outside the Kernel boundary,
- bypassing SES-Core encryption or domain binding,
- injecting randomness into Kernel evaluation,
- modifying L0-L4 logic,
- modifying stability metric definitions,
- modifying collapse or SafeMode criteria,
- allowing adapters to reinterpret internal UF structures,
- weakening reproducibility requirements,
- mixing versioned components (e.g., UF v1.4.0 with SES-Core v1.3.0).

Any implementation that violates these rules is non-compliant.

22.6 Governance of Adapters

Domain adapters MUST:

- strictly follow domain-binding rules,
- use ONLY SES Summary fields for interpretation,
- reject UF evaluations in collapse or SafeMode states,
- declare compatible UF and SES-Core versions,
- NOT reconstruct or infer internal UF operators.

Adapters serve as the legally defined boundary between UF and external systems.

22.7 Reproducibility Obligations

A conformant implementation MUST ensure:

- run-to-run determinism,
- implementation-level reproducibility,
- stable behavior under parallelization,
- consistent behavior across hardware platforms,
- deterministic structural serialization for SES-Core.

Reproducibility violations invalidate all evaluations.

22.8 Governance Summary

A UF implementation is compliant ONLY if it satisfies:

- all Kernel requirements (Sections 3–7),
- all invariants and stability properties (Sections 8–9),
- the Hardening Model (Section 10),
- the Formal Mathematics framework (Section 11),
- the Validation requirements (Sections 12–13),
- the SES–Core rules (Section 15),
- the domain adapter rules (Part IV),
- the compliance rules (Section 19),
- the reproducibility and auditability rules (Section 19),
- the versioning rules defined in this section.

UF governance ensures safety, correctness, and integrity across all domains.

Figures

- No standalone figures registered.
- All figures trace to domain sections in Parts I–IV.

Changelog

Version 1.4.0

- Renumbered Kernel Sections 3–7.
- Installed Section 10 (Hardening Model) from legacy Section 23.
- Corrected Part and Section headers throughout the specification.
- Restored Governance and Versioning Rules (Sections 18–22).
- Completed Academic Validation Layer (Section 12).
- Installed Operational Validation Suite (Section 13).
- Harmonized Section 13 main header and academic substructure.
- Corrected TOC visibility for all Sections 1–22.

Version 1.3.0

- Introduced SES-Core encryption platform.
- Added Financial Domain Adapter.
- Established preliminary adapter architecture.

Version 1.2.0

- Added Stability Metrics and Invariants.
- Expanded Decision Dynamics and Resonance Engine.

Version 1.1.0

- Added Structural Field Normalization (SEV).
- Added Gate Segmentation layer.

Version 1.0.0

- Initial release of UF-Spec.